- SQL schema creation scripts.

```
CREATE TABLE customers (
  customer_id VARCHAR(20) PRIMARY KEY,
  customer_name VARCHAR(100),
  region VARCHAR(20)
);

CREATE TABLE products (
  product_id VARCHAR(20) PRIMARY KEY,
  product_name VARCHAR(100),
  category VARCHAR(50),
  price DOUBLE PRECISION
);

CREATE TABLE transactions (
  transaction_id VARCHAR(20) PRIMARY KEY,
  customer_id VARCHAR(20),
  product_id VARCHAR(20),
  quantity INTEGER,
  discount DOUBLE PRECISION,
  total_amount DOUBLE PRECISION,
  transaction_date TIMESTAMP,

  CONSTRAINT fk_customer
    FOREIGN KEY (customer_id)
    REFERENCES customers(customer_id),
```

```
    CONSTRAINT fk_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
);
```

- SQL query scripts with sample outputs.

1. Total sales by region and category.

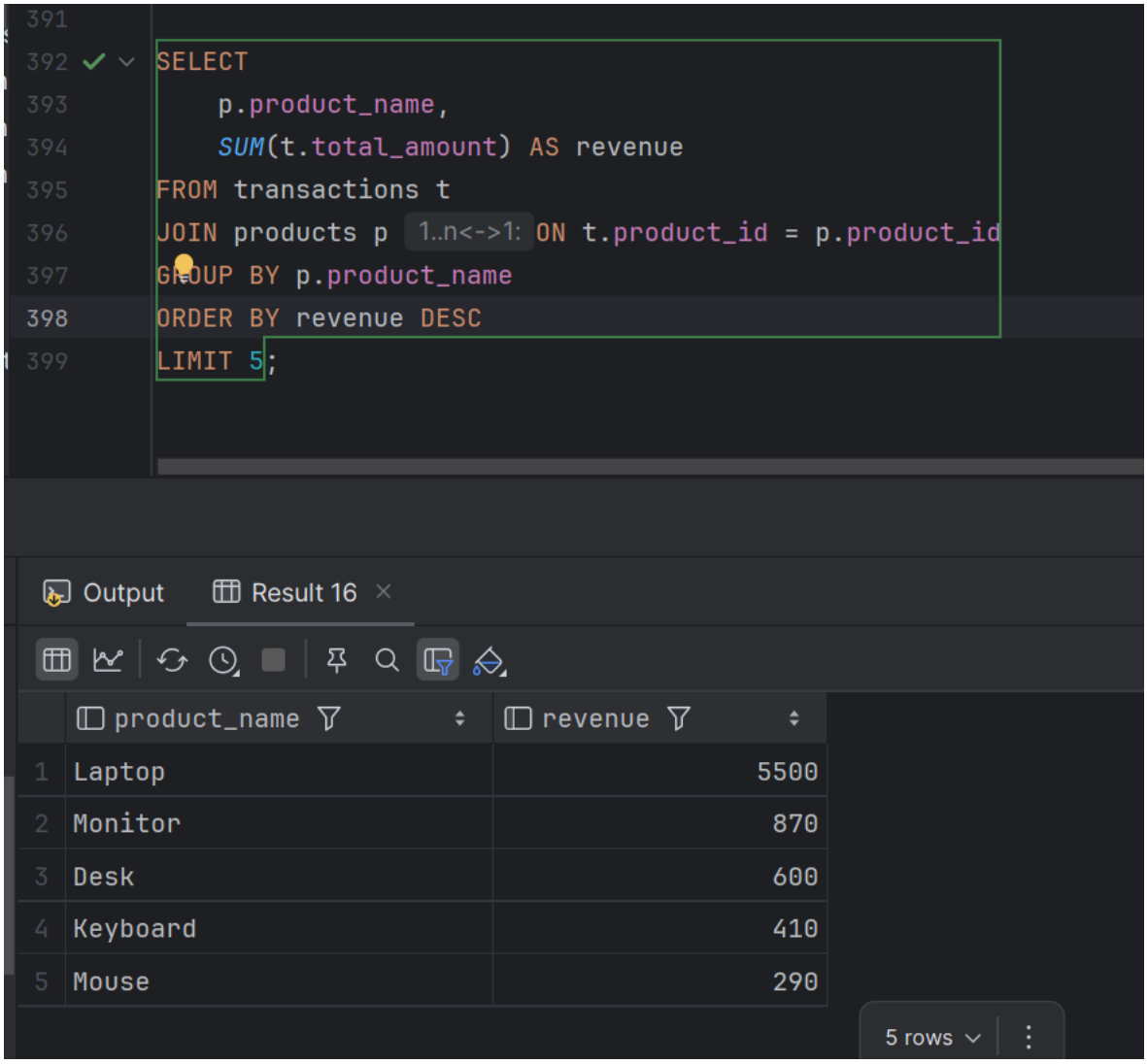
```sql
SELECT
    c.region,
    p.category,
    SUM(t.total_amount) AS total_sales
FROM transactions t
JOIN customers c 1..n<->1: ON t.customer_id = c.customer_id
JOIN products p 1..n<->1: ON t.product_id = p.product_id
GROUP BY c.region, p.category
ORDER BY total_sales DESC;
```

Output | Result 15

	region	category	total_sales
1	North	Electronics	4600
2	South	Electronics	900
3	North	Furniture	600
4	West	Electronics	600
5	East	Electronics	270
6	North	Accessories	240

9 rows

2. Top 5 products by total revenue.



3. Monthly sales trend.

```
391
392 SELECT
393     DATE_TRUNC('month', transaction_date) AS month,
394     SUM(total_amount) AS total_sales
395 FROM transactions
396 GROUP BY month
397 ORDER BY month;
```

Output | Result 17

	month	total_sales
1	2023-01-01 00:00:00.000000	1900
2	2023-02-01 00:00:00.000000	100
3	2023-03-01 00:00:00.000000	270
4	2023-04-01 00:00:00.000000	170
5	2023-05-01 00:00:00.000000	600
6	2023-06-01 00:00:00.000000	1090
7	2023-07-01 00:00:00.000000	840
8	2023-08-01 00:00:00.000000	

8 rows

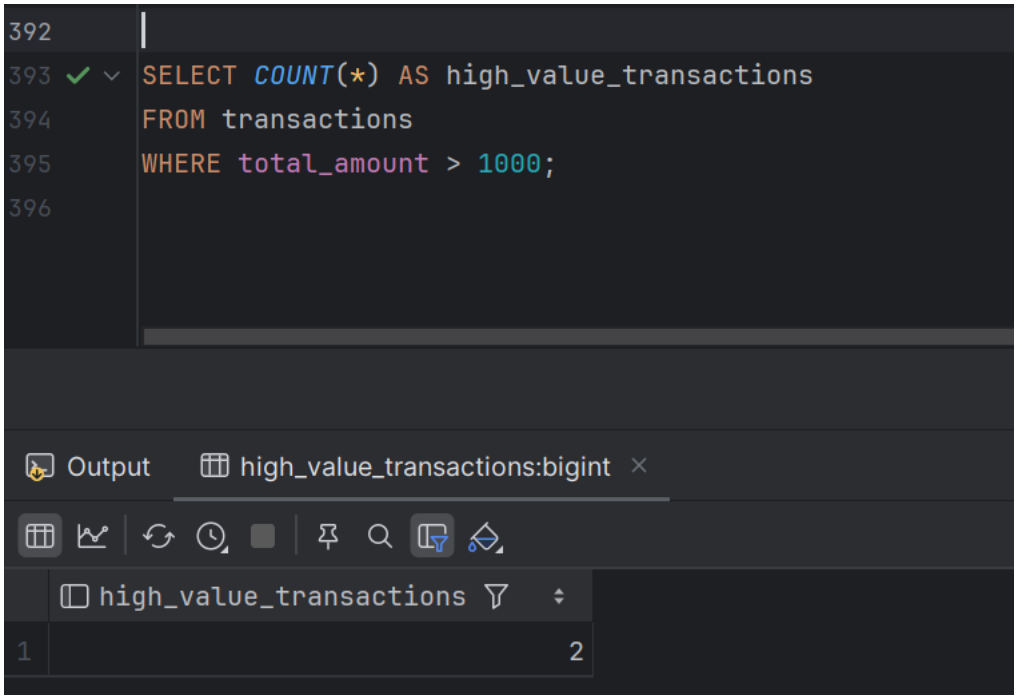
4. Average discount percentage per region.

```
391 SELECT
392     c.region,
393     ROUND(AVG(t.discount)::NUMERIC * 100, 2) AS avg_discount_percent
394 FROM transactions t
395 JOIN customers c  1..n<->1: ON t.customer_id = c.customer_id
396 GROUP BY c.region;
397
```

Output | Result 29

	region	avg_discount_percent
1	East	7.5
2	South	5
3	West	7.5
4	North	8.75

5. Number of transactions with total_value > $1000.



- Index creation and explanation of performance impact: Indexes make data retrieval faster by allowing the database to quickly locate relevant rows instead of scanning the entire table.

-- For joins

CREATE INDEX idx_transactions_customer ON transactions(customer_id);

CREATE INDEX idx_transactions_product ON transactions(product_id);

-- For filtering/grouping

CREATE INDEX idx_customers_region ON customers(region);

CREATE INDEX idx_products_category ON products(category);